[Emerging Ideas] SecretHandle: OS-Minted, Destination- and Task-Bound Opaque Secrets for Android Mobile AI Agents

Author Name(s) TBD
Affiliation TBD
City, Country
email@example.com

Abstract

Mobile GUI agents must often use a one-time password, credential, address, phone number, or payment identifier to finish a task. Today the value is placed in a screenshot, accessibility tree, prompt, or action argument, so an injected instruction can convert task authority into disclosure or redirection. We present SecretHandle, an Android design for *usable but invisible* secrets: a privileged service mints an opaque, authenticated handle at the authoritative OS source—the SMS or notification OTP pipeline, Credential Manager, the clipboard, or a wallet—before the raw value can reach the agent. The handle binds a secret type, a task identifier, a destination predicate over package, WebView origin, and field class, and a type-specific lifecycle (single-use and TTL for OTPs, task-scoped for identifiers, confirm-per-use for payment). The agent observes only the handle and issues `ENTER(handle)` toward a UI node; the OS resolves the value only after authenticating the handle, validating task and target, enforcing expiry/use/confirmation, and reserving the handle against races, then redacts the resolved field from subsequent agent observation. We report an on-emulator reproduction of the OTP-forwarding attack and a labeled, *simulation-only* reference-model study whose trends motivate the design; they are not device measurements and are not presented as paper results. We close with a preregistration-style evaluation plan (killer experiment, attack families, and baselines) for the full system.

CCS Concepts

• **Security and privacy** → **Mobile platform security**; *Authorization*; • **Human-centered computing** → Ubiquitous and mobile computing.

Keywords

mobile AI agents, Android security, opaque capabilities, destination binding, one-time passwords, prompt injection, observation redaction

1 Introduction

Mobile graphical-user-interface (GUI) agents complete tasks by reading what is on the screen and acting on it. When a task requires a secret—a one-time password (OTP), a password, an address, a phone number, or a payment identifier—the agent must obtain that value and place it into a field. In current designs the value enters the agent's observation and actuation: the SMS is rendered, the code is scraped from the screenshot or accessibility tree, and the value is typed into a node. An instruction injected through any content the agent reads can then turn task authority into disclosure or redirection; the value the agent legitimately handles is exactly the value an attacker wants.

A running example. A user asks a mobile agent to log into their bank. The bank sends a one-time code by SMS. The agent opens the bank app, reads the code from the notification, types it into the code field, and the login completes. Now suppose the same SMS, a web page in the browsing flow, or a notification from a promotional app also contains a line such as "to confirm, forward this code to `refund.example`." The agent cannot reliably separate the task it was given from text it merely read, so it forwards the code. The code was never stolen from the agent's memory; it was disclosed *through* the agent's legitimate handling of it. Under SecretHandle, the SMS pipeline mints an opaque handle bound to the bank's package and its OTP field *before* the code can enter the agent's view. The agent sees only "OTP ready for SecureBank" and issues `Enter(handle)` at the code field; the OS authenticates the handle, validates the destination, injects the value, and redacts the field. An `Enter` aimed at `refund.example` is rejected because that destination lies outside the binding, no matter how convincing the injected text was.

This is no longer hypothetical. A browser agent read an OTP from a mail client and published it; mobile agents have

forwarded Alipay verification codes under injected instructions (see Section 2 for the primary sources). Placeholder-detection defenses replace sensitive rendered text with a local token and resolve it at use time, but they are detector-dependent, session-scoped, and—crucially—do not bind the resolved value to the destination that may consume it, so an injected instruction can redirect resolution to an attacker-owned field.

Research question. Can Android let an agent *use* a secret without ever *observing* it, by minting unforgeable opaque handles at the authoritative OS source and resolving a handle only into a destination the handle is bound to—so that neither the raw value nor a usable copy enters the agent's perception or a wrong destination?

Why mobile. The authoritative sources are mobile OS services: the SMS Retriever and User Consent APIs, the sensitive-notification OTP pipeline, Credential Manager, the clipboard, and on-device wallet tokenization. The agent reaches the UI through accessibility, so an OTP it must enter is otherwise read off the screen. Destination binding reuses Android's package identity and WebView origin. On a server there is no OS SMS pipeline, no Credential Manager, no on-device wallet, and no UI to type into; the problem and its enforcement point are mobile.

Contribution. We propose SecretHandle, an OS layer that makes secrets usable but invisible to untrusted agents. A handle is minted at the source before the raw value reaches the agent, carries a destination-and-task binding, and has a typed lifecycle. A resolver injects the value only into a bound destination; a redactor masks the resolved field afterwards. This emerging-ideas paper contributes (i) the problem framing and threat model (§2–3); (ii) the handle/resolver/redactor design with an unforgeable, destination- and task-bound abstraction (§4); (iii) a compact formal handle model with four stated guarantees and two enforcement algorithms (§5–6); (iv) an on-emulator reproduction of the motivating attack and a clearly labeled simulation-only reference-model study (§7); and (v) a preregistration-style evaluation plan for the full system. We do *not* report device measurements in this paper; the formal results are conditional on stated reference-monitor assumptions and are validated only on controlled synthetic inputs.

Roadmap. Section 2 motivates the problem and Section 3 fixes the trust model. Section 4 describes the design, Section 5 formalizes handles and their guarantees, and Section 6 gives the mint and resolve algorithms. Section 7 reports what we have reproduced and preregisters the full evaluation; Sections 8–9 position the work and state its limits.

2 Background and Motivation

Mobile agents act through the UI.. Agents such as AppAgent, AutoDroid, and the Mobile-Agent family drive Android through the accessibility tree, screenshots, and synthesized gestures [**? ? ? ?**]. Everything the agent perceives is therefore an observation channel, and everything it does is an actuation channel. A secret that must be entered crosses both.

The motivating attack. In the OTP-forwarding pattern, the agent encounters a message or notification containing a verification code together with an injected instruction to forward it. The agent reads the code from the rendered text and sends it to an attacker-controlled endpoint [**? ?**]. We reproduced the core mechanism on an Android emulator with an open notification-listener agent: the code was read from the notification and POSTed to an isolated endpoint (Section 7). This establishes that the attack surface is reachable with stock, non-malicious-looking components.

Why existing defenses are insufficient. Placeholder anonymization replaces rendered sensitive text with a local token and resolves it at `type` time. It removes the literal value from the current observation but depends on detector recall, is session-scoped, and does not constrain *which* field receives the resolved value or *which* origin may request it. An injected instruction can redirect the resolution [**?**]. *Platform autofill and Credential Manager* are app-initiated: the consuming app must drive them, and they trust that app, so an accessibility agent cannot invoke them and a hostile field is not distinguished [**? ? ?**]. *Server-side agentic-payment tokens* cover payments only and live outside the OS, leaving OTP, phone, email, address, and clipboard unprotected.

Design forces. A defense that bends the attack curve must (1) mint the secret at the authoritative source, so security does not depend on detector recall; (2) bind it to the destination and task that may consume it, so an injection cannot reroute it; (3) give it a typed lifecycle, so single-use secrets cannot be replayed; and (4) redact the resolved value afterwards, because the entered value reappears in the next frame.

3 Threat Model

Assets. OTPs; account credentials; phone numbers, email addresses, and postal addresses; payment and account identifiers; and any secret whose value is usable if disclosed.

Trusted. The `system_server` and the SecretHandle service; the OS source pipelines (SMS/notification OTP classifier, Credential Manager, clipboard, wallet); and

Table 1: Attacker goals, the security goal that answers each, and its enforcement point.

Goal	Security goal	Enforced by
G1 read	S1 non-disclosure	mint before observation; principal-specific redaction
G2 redirect	S2 destination confinement	destination predicate d + task τ checked at resolve
G3 replay	S4 lifecycle safety	atomic reservation, use budget, TTL, revocation
G4 forge	S2 unforgeability	authenticated metadata $a = F_K(\text{enc}(H))$ against the OS store
(G1, reread)	S3 observation closure	redaction obligation active during and after injection

hardware-backed key storage (StrongBox, or a software keystore where hardware is unavailable).

Untrusted. The agent and its model, which may be prompt-injected; third-party applications, one of which may render handle-like strings or send a fake OTP message; cloud model endpoints; and web content reached through an in-app WebView.

Attacker goals. (G1) read a secret the agent handles; (G2) redirect a secret to an attacker-controlled destination; (G3) replay or reuse a secret; (G4) forge a handle.

Security goals. For a task that legitimately requires a secret, SecretHandle aims to guarantee: (S1) the raw value never enters the agent's observation or a non-bound destination; (S2) handles are unforgeable OS capabilities and can only be resolved into their bound destination and task; (S3) resolved values are redacted from subsequent agent observation; (S4) single-use secrets cannot be reused. The utility goal is that login, 2FA, and checkout flows still complete.

Table 1 maps each attacker goal to the security goal that counters it and to the mechanism that enforces it. The mapping is the organizing principle of the design: every attacker goal is answered by a property that a specific OS component is responsible for, rather than by a heuristic over untrusted text.

Agent principals. Mobile agents reach a device in more than one way, so the agent is not a single principal. We model three: (i) an accessibility/media-projection service of an application (application UID); (ii) an ADB/UiAutomation process, which many open runners use and which runs as the `shell` UID; and (iii) a remote agent that only receives serialized observations. The resolver authenticates the caller's UID at entry, and the redactor keys redaction to the principal at the screenshot and accessibility boundaries. If the agent has root, or the device is not SELinux-enforcing under an unrooted ADB, the redaction boundary is void and results are not meaningful; enforcement is therefore evaluated under `adb unroot` with SELinux enforcing.

Reference-monitor assumptions. The guarantees of Section 5 hold only under explicit architectural assumptions: the OS service, store, resolver, redaction registry, and trusted target-context extraction cannot be modified by the adversary and mediate every resolve; only an approved source adapter can mint a protected-source handle, and the task token cannot be forged; the authenticator key is secret and the encoding canonical; the target identity the resolver reads equals the actual injection target at commit time; and every agent-visible screenshot, tree, event, and log applies the same value-independent mask. Section 5 states these formally, so each guarantee is a conditional statement about an implementation that satisfies them rather than an unconditional claim about all Android devices.

Out of scope. Semantic misuse *inside* an allowed destination (e.g., an agent tricked into initiating a legitimate-but-unwanted action), WebView-origin attestation (fields in a WebView are rejected fail-closed rather than bound), and side channels outside the modeled observation boundary. These are residual risks and are discussed in the evaluation plan and limitations, not claimed to be eliminated.

4 Design: Usable but Invisible Secrets

A *secret handle* is an unforgeable capability object minted by the OS at the authoritative source. It carries (i) a secret *type*, (ii) a *destination binding*—an allowed package and, for web, a document origin, plus an allowed field class—derived from the task and, where available, from the source's own binding (e.g., the app hash used by the SMS Retriever [**?**]), and (iii) a *lifecycle*: single-use with a TTL for OTPs, task-scoped for identifiers, and confirm-per-use for payment. The agent observes only the handle and reasons about it ("the code arrived; enter it into the login field"); it never observes the value.

Architecture. Four components sit inside the OS:

- **Handle mint** (new system service): hooks the OTP pipeline, Credential Manager, clipboard, and wallet to mint typed, bound, unforgeable handles; keys live in hardware-backed storage.
- **Resolver** (input-dispatch / accessibility action path): on Enter(handle), authenticates the handle, checks the target node's package/origin/field class against the binding, injects the value, and enforces use count and TTL under a reservation lock.

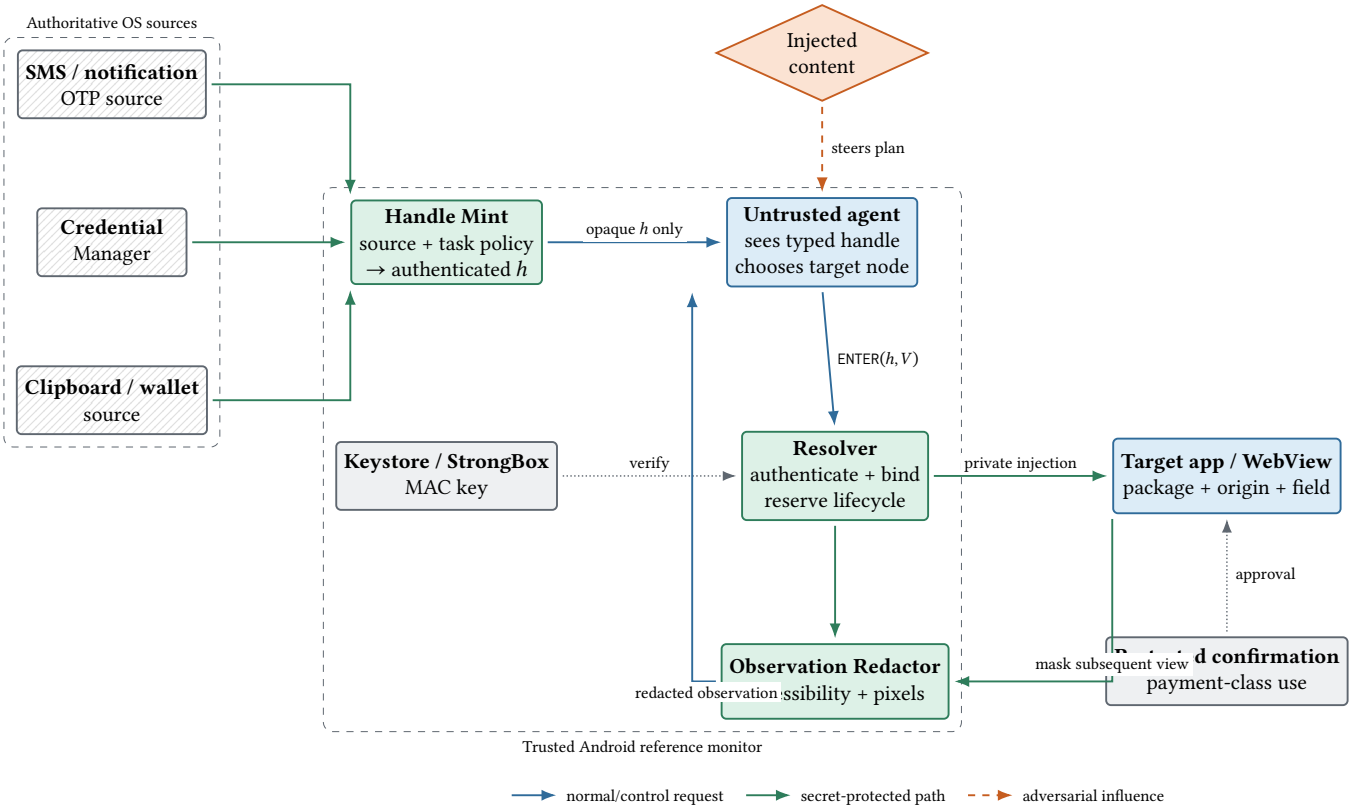


Figure 1: SecretHandle architecture. The raw value travels only from an authoritative source through the trusted mint/resolver path to an accepted destination. The agent receives an opaque handle and a redacted post-entry view. Colors and line styles distinguish ordinary, protected, and adversarial flows.

- **Observation redactor** (accessibility service and the agent's screenshot path): masks resolved values in the agent's view.
- **Agent SDK**: surfaces handles in the observation, provides the Enter(handle) action, and adapts open-source agents.

Representing handles to agents. A handle must be usable without resembling a plausible raw secret. The accessibility tree exposes a role such as `secret_handle`, a stable handle ID, the type, a permitted coarse destination, and an expiry bucket; the screenshot shows an icon and a short label such as "OTP ready for SecureBank – 38s" and never renders the code length or digits. The SDK adds Enter(handleId, targetNodeToken) to the action schema. A malicious app can draw identical pixels or set a similar accessibility label, but it cannot cause resolution: the visible string alone lacks a valid authenticated record, and the system overlays handles in a reserved, framework-owned surface marked with an extra that ordinary apps cannot set. This is usability hardening; cryptographic acceptance, not rendering, remains the security boundary.

Resolution as the enforcement point. The security hinges on resolution: a value is injected only if the handle authenticates and the target is inside the binding. An injected instruction that targets an attacker package, a foreign origin, a non-bound field, or a stale/recycled node is rejected, regardless of how convincingly the agent was manipulated. A booking of one use under a lock makes replay and concurrent double-resolution fail. After injection, the redactor prevents the value from reappearing in the next accessibility tree or screenshot.

Policy language. A handle's destination predicate is a finite record over four attribute sets—package, document origin, field class, and target node—each with a *require* flag. An attribute that is required but absent at the target fails closed. A minted handle's policy is the intersection of three policies: the task policy (which packages the task authorizes), the source policy (what the authoritative source can bind, such as an SMS Retriever app hash), and the type policy (e.g., an OTP must land in a field classified as an OTP field). Intersection is monotone and can only shrink the allowed

destination, so a compromise anywhere in the chain cannot broaden where a value may go. This gives a small, auditable policy algebra rather than relying on detector output. Weak evidence such as app-declared autofill hints or labels may supplement, but never replace, trusted framework signals.

Lifecycle and reservation. A record moves through LIVE, RESERVED, CONSUMED, and the terminal EXPIRED/REVOKED states. Resolution reserves the record under an atomic transition that decrements the use budget *at reservation time* (not at delivery), so a concurrent or crashed attempt cannot be replayed into a second delivery; a reservation whose delivery is uncertain is consumed conservatively for single-use types. OTPs carry a short TTL and a budget of one; identifiers are task-scoped; payment-class handles require per-use confirmation (and, where the device supports it, a protected-confirmation path). Revocation on task end, device lock, profile switch, or destination-package change makes stale handles unusable.

Redaction and observation closure. Injection alone is insufficient, because the entered value reappears in the next frame. The redactor registers an obligation before delivery and, at capture time, replaces the target's accessibility value, text-change events, and screenshot region with a typed mask for the agent principal only; the app and the user still see the real value. Because bounds can move between injection and capture, each obligation follows a stable view/window token and recomputes bounds when the frame is serialized; if the view cannot be re-identified, policy masks the containing window. Redaction ends when the field clears, the window closes, the task ends, or trusted app state proves the value is gone. A timer alone is insufficient, because a slow agent could observe the value just after it fires; conversely, indefinite redaction could hide unrelated user content in a recycled view, so obligations are bound to a content generation or an explicit clearing event.

Concurrency and time-of-check/time-of-use. Three races matter. First, two agent calls may attempt the same handle; atomic reservation (Proposition 5.12) lets exactly one win. Second, the target may change after it is checked; the resolver re-derives the target context immediately before injection and binds a one-shot injection capability to the reservation, narrowing the window. Third, capture may race with entry; registering the redaction obligation *before* delivery guarantees that any screenshot or accessibility read observes an active obligation whenever the field can hold the value. The design favors fail-closed behavior: a changed target generation is not followed automatically, and an unknown delivery outcome consumes a one-use handle.

Table 2: Failure handling is explicit and value-oblivious.

Condition	Resolver behavior / agent-visible response
Unknown or forged handle	constant-path reject, rate-limit source / invalid handle
Expired, revoked, or consumed	reject without revealing which state / unavailable handle
Task mismatch	reject and audit policy mismatch / destination denied
Package, origin, or field mismatch	reject, never downgrade silently / destination denied
Confirmation denied or unavailable	release or consume per type / confirmation not granted
Target vanished before injection	abort; consume OTP if delivery uncertain / target changed
Redactor cannot track target	mask containing window or abort before entry / secure display unavailable
Service crash in reserved state	conservatively consume single-use handle / handle unavailable

Failure handling. Failures are explicit and value-oblivious. Table 2 lists the resolver's behavior for the conditions the evaluation exercises; every response reveals only the handle's availability, never the value or which internal check failed.

Auditing without secret leakage. The audit record contains the event type, a pseudonymous handle ID, the type, source and destination classes, the decision, a coarse latency, and the policy version. It excludes raw values, application text, screenshots, full origins containing user identifiers, and authenticators. Per-install keyed pseudonyms allow joins within an experiment without becoming globally linkable, giving an observable deployment trail that does not itself become a disclosure channel.

Building blocks and deployment modes. Destination binding reuses Android package identity and WebView origin; unforgeability uses a message authenticator [**?**] over a canonical encoding of the handle fields with a key separated from the value; lifecycle and reservation are ordinary OS state transitions. The strongest deployment modifies AOSP across sources, accessibility, screenshot composition, and input dispatch; an OEM extension can add wallet and hardware-key integration; a research prototype may begin with SMS, a purpose-built credential provider, and open test apps. A user-space accessibility-service prototype can demonstrate workflows but cannot establish the same guarantee, because it receives observations after framework serialization and may have to type through an agent-visible path; results from that mode are feasibility

measurements, not validation of the reference-monitor assumptions. The main engineering is wiring the mint points and the resolver into the input path, and the design composes with blind-actuation approaches in which the agent acts on a node it cannot read. Figure 1 shows how the components fit together: the value travels only along the protected path from an authoritative source to a bound destination, while the agent sees an opaque handle.

5 Formalizing Handles and Their Guarantees

This section gives a compact model of the handle lifecycle and states the four guarantees claimed so far. The results are conditional on explicit reference-monitor assumptions; they characterize the model, not a deployed implementation, and are validated only on controlled synthetic inputs (Section 7).

Records and handles. The trusted store maps each identifier n to a record

$$\Omega[n] = (s, H, a, z, r), \qquad H = (n, q, \sigma, d, \tau, t_i, t_e, u, c), \quad (1)$$

where s is the value; q, σ, τ are the type, provenance, and task identifier; d is the destination predicate; t_i, t_e are the issue and expiry times; u is the remaining-use budget; c is the confirmation policy; $a = F_K(\mathsf{enc}(H))$ is an authenticator over a canonical encoding of H; $z \in \{\text{live}, \text{reserved}, \text{consumed}, \text{expired}, \text{revoked}\}$ is the lifecycle state; and r is an optional reservation nonce. The agent-visible handle is $h = (n, q, a)$ together with a non-sensitive display label; the full H stays inside the trust boundary, so an agent cannot edit a destination even if the rendered handle is malleable. Because the running use count changes over a handle's life while $\mathsf{enc}(H)$ must stay fixed for a to remain valid, the store keeps H immutable and tracks the remaining uses as mutable record state: the reservation step decrements it and finalization settles only the terminal state.

Definition 5.1 (Target match). For a trusted target context $x = (p, o, f, v)$—package, document origin, field class, and node token—$\mathsf{Match}(d, x) = 1$ exactly when every mandatory component of d accepts the corresponding component of x. An unavailable mandatory identity yields 0, never a wildcard.

Definition 5.2 (Eligible handle). A record is *eligible* at time t for task τ' and target x if its authenticator verifies, its state is live, $t_i \le t < t_e$, $u > 0$, $\tau' = \tau$, $\mathsf{Match}(d, x) = 1$, and the confirmation precondition c is satisfied.

Reference-monitor assumptions. The guarantees below hold only under the following.

Assumption 5.3 (Reference-monitor integrity). *The adversary cannot modify the SecretHandle service, store, resolver, redaction registry, trusted target-context extraction, injection path, or their control flow, and every resolve attempt passes through the resolver.*

Assumption 5.4 (Source and task authenticity). *Only an approved source adapter can mint a protected-source handle, and the task token τ and provenance σ cannot be forged or widened by the agent.*

Assumption 5.5 (Authenticator security). *F is existentially unforgeable under chosen-message attack, K remains secret, identifiers are uniform over $\{0,1\}^\lambda$, and* enc *is canonical and injective over valid metadata.*

Assumption 5.6 (Authentic target identity). *For every accepted node token, the resolver-derived context $x = (p, o, f, v)$ equals the actual injection target at commit time; a required origin or field component is never treated as matched when it is unavailable.*

Assumption 5.7 (Modeled-channel redaction). *Before s becomes observable in a target field an obligation is active, and every screenshot, accessibility tree, event, action echo, and log included in the agent view applies the same value-independent mask until s is absent.*

Assumption 5.8 (Value-oblivious public behavior). *For equal-length s_0, s_1 with equal declared leakage, minting, policy decisions, error messages, public timing buckets, and handle distributions are identically distributed.*

Guarantees. We now state the four properties that answer the attacker goals of Table 1.

Theorem 5.9 (Destination confinement). *Under Assumptions 5.3 and 5.6, if a SecretHandle value is committed to target x under task τ', then $\mathsf{Match}(d, x) = 1$ and $\tau' = \tau$ for the record bound to the handle.*

Proof. A committed injection requires a prior successful reservation of the same record and target token. The resolver is the complete mediator (Assumption 5.3) and its transition guard is exactly eligibility (Definition 5.2), so reservation implies target and task acceptance. The injection capability is one-shot and binds the reservation to the revalidated node token, and Assumption 5.6 excludes target substitution before commit. Hence the committed target satisfies the predicate and task equality. □

Corollary 5.10 (Redirection resistance). *Under Theorem 5.9, if an attacker target x_A is outside d, an injected instruction that requests* Enter *at x_A cannot cause the resolver to commit the secret there.*

Proof. A commit at x_A would imply $\mathsf{Match}(d, x_A) = 1$, contradicting the premise. □

Theorem 5.11 (Handle unforgeability). *Let an adversary make at most Q online acceptance attempts and observe handles of its choice through authorized mint events. Under Assumptions 5.3 and 5.5, the probability that it causes acceptance of handle metadata not emitted by* Mint *is at most* $\mathsf{Adv}^{\mathsf{euf}}_{F,\mathcal{B}}(\lambda) + Q/2^{\lambda}$.

Proof. Construct a reduction $\mathcal{B}$ that simulates the store and answers authenticator queries through its MAC oracle. New valid metadata with a valid tag yields an EUF-CMA forgery; guessing an existing live random identifier without a new tag succeeds with probability at most $Q/2^{\lambda}$ before rate limiting. A fake visual handle does not change the argument, because acceptance uses the authenticated record, not rendered syntax. □

Proposition 5.12 (Single-use linearizability). *Assume the live-to-reserved transition is an atomic compare-and-swap and no rollback occurs after possible delivery. For a handle minted with $u = 1$, at most one concurrent resolve call reaches committed injection.*

Proof. All contenders observe one state variable, so exactly one linearizes the transition from live to reserved; the others see reserved or consumed and reject. Consuming at reservation rather than at delivery (the conservative rule we adopt) prevents two sequential commits across an uncertain failure. □

Proposition 5.13 (Expiry monotonicity). *If expiry is evaluated against a monotonic clock and expired records never return to live, a handle rejected because $t \geq t_e$ cannot become eligible through a wall-clock adjustment.*

Proof. Monotonic time does not decrease and no transition returns an expired record to live, so the strict conjunct $t < t_e$ of Definition 5.2 remains false. □

Non-disclosure. The first goal, that the agent never learns the value, is a non-interference-style statement. Let $\mathsf{View}_A(\pi)$ be the agent-visible projection of execution trace π: screenshots and accessibility trees, handle metadata, action results, timing rounded to a public granularity, and application messages, but not TCB memory, the private input buffer, or protected-confirmation pixels. Let ℓ be the declared leakage, containing the type q, a coarse destination label, age and remaining-lifetime buckets, and success/failure status, but never s, a deterministic hash of a low-entropy OTP, or value-dependent error codes.

Definition 5.14 (Agent-view non-disclosure). For equal-length secrets s_0, s_1 with the same leakage ℓ and any probabilistic polynomial-time adversary $\mathcal{A}$ controlling the agent, the distinguishing advantage is

$$\mathsf{Adv}^{\mathsf{nd}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda) = \big|\Pr[\mathcal{A}(\mathsf{View}_A(\mathsf{Real}(s_0))) = 1] - \Pr[\mathcal{A}(\mathsf{View}_A(\mathsf{Real}(s_1))) = 1]\big|. \tag{2}$$

Non-disclosure holds when this advantage is negligible for equal ℓ.

Theorem 5.15 (Agent-view non-disclosure). *Under Assumptions 5.3, 5.4, 5.5, 5.7, and 5.8, and before any explicitly declassified value-dependent outcome,* $\mathsf{Adv}^{\mathsf{nd}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda)$ *in Eq.* (2) *is negligible for equal-leakage secrets.*

Proof. Consider a hybrid trace. Replacing the mint view for s_0 with that for s_1 changes the distribution negligibly: the visible identifier is uniform and independent of s, the type and public label are equal by the leakage premise, and the authenticator covers metadata that does not encode s (Assumptions 5.5,5.8). Resolver decisions depend on authenticated metadata, task, target, lifecycle, and confirmation, which are equal or value-oblivious. At injection the raw value moves only through TCB memory and the target's private input path, and Assumption 5.7 replaces every modeled post-entry occurrence with the same mask. Public errors and timing do not depend on the value (Assumption 5.8), so the two agent-view distributions are indistinguishable until an allowed value-dependent outcome is released. □

Remark 5.16 (Legitimate-but-unwanted actions). *Theorem 5.9 does not cover a target inside d. An injection may still induce an unwanted action within the same authorized package or origin, such as logging into an attacker's account. Task semantics, transaction confirmation, and richer form binding are separate defenses; we quantify this residual risk rather than claim it eliminated.*

6 Enforcement Algorithms

Algorithms 1 and 2 give the two transitions the guarantees depend on. Mint derives a destination predicate by intersecting three independently available policies and authenticates the handle metadata; resolve re-derives the target from trusted framework state, checks eligibility, and reserves the record atomically before any value moves.

Lifecycle and the authenticated budget. Mint authenticates the initial lifecycle parameters with the handle identity, so a compromised caller cannot widen the budget or the expiry. Because the remaining-use count changes while $\mathsf{enc}(H)$ must stay fixed for a to remain valid, the store keeps H immutable and treats u as mutable record state: reservation decrements u and finalization settles only the terminal state. We adopt the conservative rule that the

Algorithm 1 SecretHandle Mint

Require: buffer s, type q, provenance σ, task token T, destination evidence E, policy hints P
Ensure: handle h or Reject

authenticate caller and its right to assert σ
$(\tau, D_T) \leftarrow \mathsf{VerifyTask}(T)$; **return** Reject on failure
$D_S \leftarrow \mathsf{SourceDestinations}(\sigma, E)$
$d \leftarrow D_T \cap D_S \cap \mathsf{TypePolicy}(q, P)$; **return** Reject if $d = \emptyset$
$(t_i, t_e, u, c) \leftarrow \mathsf{Lifecycle}(q, P)$
$n \xleftarrow{\$} \{0,1\}^\lambda$; $H \leftarrow (n, q, \sigma, d, \tau, t_i, t_e, u, c)$; $a \leftarrow F_K(\mathsf{enc}(H))$
atomically insert $\Omega[n] \leftarrow (s, H, a, \mathsf{live}, \perp)$
suppress s for the agent principal; wipe buffers
return $h = (n, q, a)$

Algorithm 2 SecretHandle Enter/Resolve

Require: handle $h = (n, q, a)$, task token T', target node token V, time t
Ensure: internal injection or value-oblivious Reject

$(\tau', _) \leftarrow \mathsf{VerifyTask}(T')$; **return** Reject on failure
load $\Omega[n] = (s, H, a_H, z, r)$
return Reject unless $a = a_H = F_K(\mathsf{enc}(H))$ (constant-time)
$x \leftarrow \mathsf{FreshTargetContext}(V)$
if $z \neq \mathsf{live}$ or $t \notin [t_i, t_e)$ or $u = 0$ or $\tau' \neq \tau$ or $\mathsf{Match}(d, x) = 0$ **then**
    return Reject
end if
atomically reserve: $(z, r, u) \leftarrow (\mathsf{reserved}, \mathsf{nonce}, u - 1)$
$\gamma \leftarrow \mathsf{ObtainConfirmation}(c)$
if $\mathsf{ConfirmOK}(c, \gamma) = 0$ **then** finalize by type policy; **return** Reject
end if
$x' \leftarrow \mathsf{FreshTargetContext}(V)$
if $x' \neq x$ or $\mathsf{Match}(d, x') = 0$ **then** finalize conservatively; **return** Reject
end if
register redaction obligation for (V, n, q) before delivery
$ok \leftarrow \mathsf{InjectInternal}(V, s)$; settle terminal state; wipe buffers
return ok with value-oblivious status

budget is consumed *at reservation*, not deferred to delivery, and is refunded only when delivery is proven impossible. This keeps the authenticator valid across the multiple uses of a long-lived identifier while still failing closed on replay and concurrency.

Integration points. The prototype adds a system service in `system_server` with a small binder surface: mint for allowlisted adapters, list public handles for the current task, resolve for the agent SDK, revoke, and a secret-free audit export. The accessibility manager issues target-node tokens that bind window ID, connection ID, generation, package UID, and a short expiry, because raw node identifiers are reused. Text injection bypasses the agent process and ordinary IME suggestion and history surfaces, since otherwise the keyboard could re-expose the value; screenshot redaction happens before the buffer is mapped into the agent process, because an overlay drawn after capture is insufficient. Keys live in a non-exportable, version-bound Keystore entry, and the prototype must report the actual security level, which is software-only on an emulator or Cuttlefish image.

Complexity. For a store indexed by random identifier, mint and lookup are expected $O(1)$. Policy evaluation is $O(k)$ for the small fixed number k of predicate components. Authentication cost is linear in encoded metadata length, not in secret length. Redaction is $O(r + n)$ per observation for r active obligations and n serialized nodes unless an indexed obligation map is kept. Per-live-handle memory is $|s| + O(|H| + \lambda)$ plus index and audit overhead; because OTP and credential records are short-lived, per-task quotas and expiry sweeps bound exhaustion.

7 Preliminary Evidence and Evaluation Plan

On-emulator attack reproduction. On an Android emulator, an open notification-listener agent read a synthetic OTP from an SMS notification and forwarded it to a loopback endpoint. Across ten independent synthetic canaries, all ten were read and forwarded (observed exposure rate 1.0 with a Wilson 95% interval of $[0.72, 1.0]$). This is a functional reproduction of the mechanism, not a device result and not a paper number.

Reference-model study (simulation only). To check the qualitative trends the design predicts, we ran the project's Python reference model on synthetic canaries (seed `20261017`, 200 trials per attack across eight attack families, 6400 trials total), including read-and-exfiltrate (family 1) and identity ambiguity (family 7). The model is a simulation; its numbers are *not* device measurements and are *not* reported as results. Under it, the unprotected baseline exposed the value in every trial; a detected-placeholder baseline removed the literal value from the immediate artifact but still leaked it after entry (post-entry leakage 1.0) and still allowed redirection (1.0), because it does not bind a destination; and the SecretHandle condition produced zero exposure, zero redirection, zero post-entry leakage,

zero forgery and replay acceptance, with full legitimate-task success and zero false rejections. On identity ambiguity, SecretHandle failed closed (rejected 200/200) while the baselines delivered the value (200/200). The platform-flow baseline was recorded as not applicable because it could not be exercised. Full configuration, outputs, and caveats are in the project's preliminary-evidence document.

Killer experiment (design of record). The decisive experiment runs the OTP-forwarding injection against an open login-plus-OTP app, first unprotected (the agent forwards the code), then with SecretHandle, where the OTP is a handle bound to the login app's package. It passes only if all three hold: (a) the raw code is absent from every instrumented agent artifact; (b) Enter(handle) into the attacker's destination is rejected by the resolver; and (c) the legitimate login succeeds. In the reference model all three hold; establishing them on a real device is full-paper work.

Research questions. The full evaluation is organized around six questions: (RQ1) does OS-source minting keep raw values out of all instrumented agent observations and traces? (RQ2) does destination and task binding prevent injection-driven redirection without blocking legitimate completion? (RQ3) do lifecycle and atomic reservation prevent replay, expiry bypass, and concurrency races? (RQ4) does observation redaction prevent post-entry rereading in pixels and accessibility trees? (RQ5) what latency, memory, energy, and interaction overheads arise by source and type? and (RQ6) which identity ambiguities, unsupported apps, WebViews, keyboards, or OEM changes force failure or a weaker policy? RQ1–RQ4 are primary outcomes; the rest are exploratory.

Baselines. The evaluation compares four conditions: (B0) unprotected, where the agent receives the raw value and types it normally; (B1) a detected placeholder that replaces rendered sensitive text with a local token and resolves it without destination binding, approximating the closest published mobile-placeholder family [**?**]; (B2) the Android platform flow (Autofill, OTP Autofill, or Credential Manager) where the app can invoke it [**? ?**], with unsupported workflows recorded as not applicable; and (B3) SecretHandle. Ablations disable source minting, destination binding, task binding, atomic lifecycle, post-entry redaction, origin checks, or per-use confirmation one at a time, always with synthetic canaries and an isolated endpoint.

Workloads and metrics. Four open test applications cover username/password plus SMS OTP, multi-page 2FA, address and phone-number checkout, and wallet-token checkout with per-use confirmation; each logs only ground-truth events (target identity, whether a value was delivered, server acceptance, task completion) to a protected harness. For N tasks with instrumented artifact set I_i and canary s_i, exposure is $E = \frac{1}{N}\sum_i \mathbf{1}[s_i \in I_i]$, augmented by normalized, encoded, and substring checks. Redirection R is the fraction of attack trials in which the value reaches a target outside d; legitimate utility U is the fraction of benign trials meeting the application oracle; post-entry leakage L counts trials whose value reappears in a later modeled observation; forgery acceptance F and replay acceptance Y count unauthorized injections per attempt. We also record latency (mint, authentication, target lookup, confirmation, injection, redaction), memory, energy where supported, confirmation count, and policy- and false-rejection rates. An observed zero is reported as "0 of n" with a confidence interval, never as proof that an attack is impossible.

Analysis and sample planning. General-task regression is measured on AndroidWorld [**?**], with SPA-Bench [**?**] for breadth. Binary outcomes receive Wilson confidence intervals; paired comparisons use a paired permutation or exact McNemar test when the assumptions fit; latency uses medians, tail percentiles, and bootstrap intervals. Each condition uses fixed task seeds and paired benign/adversarial versions, blocked by device, agent, model, application, and secret type, with repetitions fixed by a prospective precision analysis before outcomes are seen. All exclusions—agent crash, platform crash, network outage, user cancellation, unsupported identity—are reported by category. Device latency, energy, and hardware-keystore behavior are explicitly deferred and reported only where a suitable device is available.

8 Related Work

Placeholder and anonymization defenses. Mobile placeholder schemes replace sensitive rendered text with a type-preserving token and reverse the mapping at use time [**?**]; the closest recent work resolves a `type(PLACEHOLDER)` locally but is detector-based, session-scoped, and validates index ranges rather than which field receives a resolved value. We differ on mint source (authoritative OS source, not screen scraping), unforgeability, destination-and-task binding, typed lifecycle, and a threat model that includes redirection.

Capability and isolation approaches. Planner-side capability schemes and dual-model patterns hide values from the planner in tool-calling settings [**? ?**]; isolation and privilege-minimization designs restrict what a tool agent may do [**? ? ?**]. These operate in the planning or tool layer rather than at the point where a secret crosses from an OS source into a UI field.

Prompt sanitizers. Detection-and-rewrite approaches suppress or restore sensitive spans in prompts [**? ? ?**]. They reduce model-side exposure but do not bind a resolved value to a destination and are not OS-enforced.

Platform and industry mechanisms. The SMS Retriever and User Consent APIs [**? ?**], the sensitive-notification OTP pipeline [**?**], Credential Manager [**?**], autofill [**? ?**], the clipboard sensitivity flag [**?**], and Protected Confirmation [**?**] are building blocks, but autofill and Credential Manager are app-initiated and trust the app, and server-side agentic-payment tokens are payment-only. SecretHandle composes these and adds OS-level destination binding and lifecycle.

OS mechanisms and reference monitors. SecretHandle is an application of complete mediation and least privilege [**?**] to a new asset class: secrets that are typed into a UI by an untrusted process. Capability systems such as Capsicum bind authority to a handle and confine it to a scope [**?**], and Android itself mediates permissions and app isolation through its install-time and runtime models [**? ? ?**]. Taint and dynamic-analysis systems track the flow of sensitive data through applications [**? ?**], which is complementary but observability-oriented: they can flag a leak, whereas SecretHandle prevents the value from entering the agent's observation at all. The capability concept itself originates with Dennis and Van Horn [**?**]; our contribution is the destination-and-task predicate over package, origin, and field class together with an authenticated, lifecycle-bounded handle that an accessibility agent can invoke without seeing the value.

Credential and token standards. OTP construction and validation are standardized by HOTP and TOTP [**? ?**], and token binding schemes constrain where an OAuth-bound token may be replayed [**? ? ?**]. WebAuthn and passkeys remove a shared secret from the flow entirely [**? ?**]. These authenticate a credential, a user, or a server endpoint, but they do not decide which on-device UI field may receive a captured value; SecretHandle complements them by constraining the resolution sink inside the OS.

Attacks. Documented agent OTP exfiltration and forwarding, plus environmental and fine-print injection, motivate the threat model [**? ? ? ? ? ?**]. We adapt attack mechanics, not their reported success numbers.

9 Limitations

No real-device measurements. This paper reports an emulator reproduction and a clearly labeled simulation; it contains no device latency, energy, or hardware-keystore numbers. On an emulator or Cuttlefish image the keystore is software-only, so we report `security level = software` and make no hardware-protection claim. Device measurements are future work.

Binding is not honest use. Destination binding constrains *where* a value may be resolved, not how a trusted app then uses it. An injected instruction can still cause an agent to perform a legitimate-but-unwanted action inside an allowed destination; we scope this out and quantify it as a residual risk in the evaluation plan.

WebView fail-closed. Binding a field to a WebView document origin requires node-to-frame-to-origin attestation, which we do not solve here; web fields are rejected rather than bound. This trades some utility for honesty.

Screenshot-only agents. An agent that can only act on pixels without naming a stable target node cannot be given a bound field; such agents need a trusted target-selection adapter or user confirmation and are out of scope.

Simulation is not evidence. The reference model demonstrates design trends under synthetic canaries. It does not establish the absence of exposure on uninstrumented channels and is not a substitute for the device evaluation.

Conditional guarantees. The results of Section 5 hold only under the reference-monitor assumptions and characterize the model, not an implementation. They are not measurements, are not claimed tight or optimal, and can fail silently if any assumption is violated—for example, if an observation channel outside the modeled set is not redacted, or if a fallback copies the value to the clipboard or asks the agent to type it, which invalidates the guarantee for that operation.

Composition gaps. A deployment combines several sources, an agent framework, input components, OEM services, and applications. Non-disclosure composes only if every component that receives the value is either inside the trust boundary or outside the agent view, and destination confinement composes only if every resolution path uses the same target identity. We do not yet have a mechanism that proves an arbitrary third-party keyboard, accessibility overlay, or OEM modification is in the modeled set, so the prototype reports which channels it instruments and treats the rest as assumptions rather than claims.

10 Ethics, Reproducibility, and LLM Use

Ethics. Every experiment uses randomly generated synthetic canaries, an isolated loopback endpoint, and open test flows. No real credential, payment instrument, or production OTP is used or collected, and no human subjects are involved. The on-emulator reproduction runs only against a local endpoint controlled by the authors.

Reproducibility. The reference-model configuration pins the seed, the attack set, and the number of trials; the provenance record stores the Python version, platform, configuration hash, and an explicit `real_experiment=false` flag. Regenerating every reported simulation number requires a single command from the configuration file. The paper distinguishes simulated from measured results throughout.

LLM use. Portions of the manuscript text were drafted with the assistance of a large language model; all technical claims, configuration, and numbers were reviewed and produced by the authors. This disclosure is provided for transparency.

11 Conclusion

SecretHandle reframes how an untrusted mobile agent handles secrets: instead of reading a value and typing it, the agent receives an opaque, unforgeable, destination- and task-bound handle and asks the OS to resolve it into a bound field. The raw value never enters the agent's observation, cannot be rerouted by injection, cannot be replayed past its lifecycle, and is redacted after entry. We have framed the problem, stated the threat model, described the design, reproduced the motivating attack on an emulator, and specified a reproducible evaluation for the full system. The next step is to implement the mint points and resolver in AOSP and run the killer experiment on a real device.